

Decline Decoder

Every failed payment is a decision. Make it the right one.

 [Try the live tool](#) · [Browse the 60-scenario eval set](#) · [See the results](#)

1. The problem

Payment decline codes are cryptic, inconsistent across payment methods, and often don't tell a merchant what to do next.

A code like `05 – Do Not Honour` is a good example. Depending on the payment method, attempt history, and other transaction context, the right action could be to retry, ask the customer to contact their bank, try another payment method, or stop retrying altogether. Today, support teams often have to interpret these codes manually, while automated systems may rely on simple retry rules that treat very different failures in the same way.

Getting this decision wrong has a cost either way - retrying a hard decline can waste processing costs and lead to excessive retries, while giving up on a recoverable decline means losing revenue that could have been recovered.

The scale is significant. UPI AutoPay alone processed 808 million debits in July 2025, and around 74% of rejections were business declines.

Having worked with payment failures for years, I wanted to explore whether AI could turn these messy signals into a clear decision: what happened, what should the merchant do next, and what should they tell the customer?

2. What I built

Paste a decline code and a few details—payment method, amount, one-off or recurring, attempt number, and gateway message. Decline Decoder gives you:

- **Root cause** — what went wrong, in plain English
- **Retry guidance** — whether to retry, when to retry, or wait for customer action
- **Customer recovery message** — a message tailored to the failure type, the customer's payment history, the channel, and the attempt number, rather than a generic message

- **Prevention step** — what the merchant can do to reduce the same failure in the future

Upload a batch of failures and it does the same at scale: how much is recoverable now, and which repeated problems to fix and insights to handle them both reactively and proactively.

It's built for **merchant support and payment-ops teams**—use it for one-off troubleshooting or in bulk to improve retry rules and prevent repeat failures.

3. Why an LLM instead of a lookup table?

For mapping a code to its cause, a lookup table is better: it is more accurate, deterministic, and auditable. On my test set, the lookup-based approach got **88% of retry decisions right**, compared with **70–74% for the LLM alone (v1)**. The LLM also invented answers for 4 of 6 unknown codes.

But a lookup table can only tell us **what the code means**. It cannot use the full payment context—such as attempt number, amount, recurring vs. one-off, or whether the customer was already debited—to determine the right recovery action.

The table provides the trusted facts; the LLM applies those facts to the payment context. In v2, this combination increased retry-decision accuracy to **98%** and turned the decoder from a code-lookup tool into a recovery assistant.

It can now recommend **whether and when to retry, what the merchant should do next, how to reduce repeat failures, and an adaptive customer message tailored to the failure, payment history, channel, and attempt number.**

4. Eval design

I built the test set before the product: 50 development scenarios (30 clear, 8 ambiguous, 7 traps, 5 unknown codes) and 10 held out, each with an answer key for root cause, retry decision, timing and customer action, grounded in NPCI, RBI, card-network and gateway documentation. **Primary metric:** retry-decision accuracy. **Guardrail:** hallucination on unknown codes, target zero.

* **The holdout set.** The 10 holdout scenarios were written alongside the development set and were not looked at during prompt development. v2 was run against them once, after the prompt was frozen.

	Lookup table only	v1: AI only	v2: lookup + AI (dev, n=50)	v2: lookup + AI(holdout, n=10)
Retry-decision accuracy	88%	70–74%	98%	100%
Unknown-code hallucinations	0/5	4/5	0/5	0/2
Root-cause accuracy	96%	82–84%	100%	100%
Message-rule breaches	—	13/50	9/50	0/10
Latency / cost per diagnosis	instant / ₹0	3.3 s / ~₹0.3	7.2 s / ~₹0.8	7.2 s / ~₹0.8

* 8 out of the 9 are SMS length overruns — a correct message that exceeds the 160-character budget — which makes this a missing validation step rather than a reasoning failure.

5. What failed, and the guardrails that fixed it

v1 failed in two ways: it **guessed when it didn't know** (made-up answers for 4 of 6 unknown codes), and it **ignored context**, such as retrying after a third failed attempt or overriding a Mastercard instruction to wait 24 hours. The guesses were its most expensive mistakes, because a merchant could act on a confident wrong answer.

How v2 prevents these:

- **Unknown codes never reach the AI.** Every code is checked against the reference first. If it isn't found, the tool says *"Not recognised"* and makes no AI call.
- **The AI can't invent a cause.** For known codes, it can only choose from causes allowed by the taxonomy for that code and payment method.
- **LLM reasoning is grounded in explicit rules.** Rules determine how factors such as attempt count, one-off vs. recurring payments, network retry instructions etc. change the recommendation.

- **Output is structured.** Every response follows the same fixed format, making it more reliable and easier to validate automatically.
- **High-risk messages use approved templates, not free-text AI.** For fraud, lost/stolen cards, or unclear debits, the model selects a pre-approved template and fills only safe fields like amount, date and reference. Message rules are enforced in code, not just in the prompt.

6. Trade-offs I made

Safety vs. coverage. Restricting the tool to known codes reduces hallucinations and the risk of costly incorrect payment diagnoses. The trade-off is coverage: new decline codes cannot be diagnosed automatically and require manual updates to the reference.

Accuracy vs. speed and cost. As we push for higher accuracy, both latency and cost increase—v2 takes 7.2 seconds and costs ~₹0.8 per check. That's acceptable for a support team, where the decoder is used after a payment fails, but too slow and expensive to run in the live payment path, where every failed transaction needs a near-instant response.

Fixed rules vs AI judgement. Code-based rules are predictable but rigid. AI can apply the same rules more flexibly and adapt to context—which can help with unusual cases, but can also lead to inconsistent interpretations. That flexibility is both the strength and the risk of using AI for rule-based decisions.

Human control vs. automation: Keeping a human in the loop improves reliability by catching incorrect diagnoses, but adds time and operational effort. Full automation is faster and scalable, but a wrong diagnosis can directly create payment risk. The right approach would be : start with control, earn the right to automate—use human review to build evidence and confidence, then automate only the cases that consistently prove safe.

7. What I'd measure, and when I'd kill it

What I'd track

- **Agent disagreement:** how often agents flag a recommendation as wrong, by cause, using the tool's "Flag this answer" button. This shows where the tool is wrong.

- **Handling time:** how long an agent takes to resolve a decline ticket with the tool, compared with without it.
- **Money recovered:** once retry timing follows the tool's recommendation, rupees recovered compared with the gateway's default schedule, ideally by splitting declines between the two, plus retries sent on "don't retry" declines, which should be zero.
- **Insight follow-through:** after a merchant acts on an insight, whether that failure type falls the next month (for example, fewer expired mandates after renewal reminders).
- **Unknown codes:** the share of declines not recognised, and how quickly new codes get added.
- **Message edit rate:** the share of generated customer messages an agent rewrites before sending, split by cause.

When I'd kill it

- **Agents don't trust it.** If 25%+ of recommendations are marked wrong after four weeks, the system isn't reliable enough to automate.
- **It doesn't recover more money.** If its retry recommendations don't outperform the gateway's default retry schedule, the core value isn't proven.
- **It doesn't know enough.** If 10%+ of real declines are returned as "not recognised," the reference data is too limited.
- **Insights don't drive action.** If merchants don't act on the insights—or acting on them doesn't reduce failures—we're not actually helping prevent them.